

ConferenceCheck Mobile: Analytics, Meals and Notifications: Validation Report

Project: ConferenceCheck Mobile: Analytics, Meals and Notifications (component 4.3b) **Student:** Ndashi Bwalya Chisanga, computer number 2021470105 **Supervisor:** Mr. Mofya Phiri
Repository: <https://github.com/ndashi-chisanga6/ConferenceCheck-Mobile-Analytics-Meals-and-Notifications> **Validated state:** tag milestone-10-evaluation

This document reports the validation results and describes the path that produces them: the seeded dataset, the commands that reproduce every figure, and the acceptance criteria each run is held to (proposal Section 5, Table 3). Every number below is taken from `tools/evaluation-results.json` or from a named test/command output, and the source is named alongside it.

Conventions. Latencies are quoted in milliseconds to one decimal place. Counts and rejection rates are exact. All timings are wall-clock, from a single named run on a local PHP 8.5.8 / PostgreSQL 18 stack, and the run is always named.

Contents

Section	Subject	Page
0	Headline results	2
0.1	Auditable evidence	2
1	Meal voucher redemption correctness	4
2	Session attendance and capacity correctness	5
3	Latency against the proposal's targets	5
4	Analytics dashboard freshness	6
5	Export correctness	6
6	Notification delivery	7
7	Baseline comparison, and its limits	7
8	Reproducibility	8
8.1	The environment these commands assume	8
8.2	Commands	8

0. Headline results

The evaluation exercise seeds a 500-attendee event with three meal categories, a voucher issued per attendee per category for 1,500 vouchers in all, and ten sessions (`tools/seed-evaluation-event.php`), then drives it through the real API (`tools/run-evaluation.py`) rather than through unit-level mocks.

Measure	Result	Proposal threshold
Duplicate redemption rejection, sequential re-scans	100/100 (100%)	100%
Duplicate redemption rejection, concurrent races	20/20 races, exactly one success each	100%
Cross-category independence	20/20 attendees redeemed all 3 categories	all categories independent
Voucher scan latency, median / p95 (n=200)	187.5 ms / 286.7 ms	< 500 ms / < 1,000 ms
Session scan latency, median / p95 (n=40)	174.5 ms / 266.5 ms	< 500 ms / < 1,000 ms
Session duplicate rejection	20/20 (100%)	100%
Dashboard/analytics freshness, worst case	≈ 30.2 s (30 s poll + 211.6 ms endpoint)	≤ 30 s
Export correctness	exact (780/780 rows across both reports)	exact

Every committed acceptance criterion in Table 3 is met, with margin, on this run. The narrowest margin is dashboard freshness, which is bound by the 30-second polling interval documented as a deliberate trade-off in the proposal's limitations (Section 8), not by anything measured here.

Source: `tools/evaluation-results.json`, produced by `python tools/run-evaluation.py` against `tools/seed-evaluation-event.php`.

0.1 Auditable evidence

Validated state. Tag `milestone-10-evaluation`. The exact commit it resolves to is verifiable with:

```
git rev-list -n1 milestone-10-evaluation
```

(`milestone-10-evaluation` is an annotated tag; a bare `git rev-parse` on it returns the tag object's own hash, not the commit, so use `rev-list -n1` or `rev-parse milestone-10-evaluation^{commit}` to resolve to the commit.)

The tag table in docs/evidence_index.md records the same mapping and what is present in that tree.

Test suite. Command and output, run on PHP 8.5.8 with PostgreSQL 18:

```
$ php artisan test
{"tool":"phpunit","result":"passed","tests":55,"passed":55,"assertions":213}
```

PHPStan level 7: clean. Pint: clean. Flutter: flutter analyze no issues, flutter test 19/19 passed.

Independent re-verification, 22 September 2026. Every command in Section 8 was re-run from the working tree on that date, on PHP 8.5.8 with PostgreSQL 18.2, to confirm the claims in this report hold rather than resting on an earlier run. Results: backend suite 55 passed / 213 assertions; PHPStan level 7 zero errors; Pint clean; the seed produced 500 attendees, 3 meal categories, 1,500 vouchers and 10 sessions; the evaluation round reported 200 of 200 scans accepted, 100 of 100 sequential duplicates rejected, 20 of 20 concurrent races resolved to exactly one success, 20 of 20 attendees redeeming all three categories independently, 20 of 20 duplicate session scans rejected, and meals.csv 280/280 with attendance.csv 500/500 against the database. Latency varies with machine load across runs; Section 3 states the envelope. The live Firebase path was exercised the same day: an organiser-targeted send reached Google, which answered with a structured FCM v1 error for the deliberately invalid probe token, and the service pruned that token, confirming that service-account signing, the Google OAuth token exchange and the authenticated FCM v1 endpoint all work from this machine today.

Adversarial and live checks, from the full strict review (docs/STRICT-REVIEW.md):

Verification	Result
Backend test suite (php artisan test)	55/55 passed, 213 assertions
Live API exercise, all endpoint groups	all responded correctly, incl. 401/403 enforcement
Double meal-voucher scan	rejected (DB transaction + lockForUpdate)
Duplicate check-in / session scan	flagged duplicate: true / rejected
Over-capacity session scan	recorded with over_capacity, warning, and an automatic organiser alert dispatched
Notification send to all_attendees	recipient records created, per-recipient delivery status recorded
Four CSV report downloads	all well-formed, row counts exact against the database
flutter analyze	no issues
flutter test	19/19 passed

Route coverage, and which routes are this component's. The backend exposes 47 API routes (php artisan route:list --path=api), distributed as: MealController 10, AttendeeController 8, SessionController 7, NotificationController 5, EventController 5, AnalyticsController 4, ReportController 4, AuthController 4.

This repository hosts the **shared** CMS v4 backend, so not all 47 routes are this component's contribution, and the count should not be read as though they were. Two of the eight AttendeeController routes, POST /events/{event}/attendees/check-in/scan and POST /events/{event}/attendees/{attendee}/check-in, are check-in capture, which is component 4.3a's remit; they are present because the backend is shared, and this component consumes the check-in data they produce as a read-only input to analytics rather than claiming them as delivered work. AuthController and EventController are likewise shared scaffolding used by both components. The routes this component contributes are the meal voucher, session attendance, notification, analytics and reporting groups, plus GET /events/{event}/attendees/me, which was added here so the attendee QR screen could fetch a real server-issued token.

The boundary is visible in the mobile codebase, which is this component's alone: mobile/lib/features/ contains dashboard, meals, sessions, notifications, reports, attendee, events, auth, profile and sync, and no check-in capture module. The only mobile code that mentions check-in at all is in features/dashboard/, where check-in counts are read back from the analytics endpoints. That is consumption, not capture.

1. Meal voucher redemption correctness

Single-use redemption is enforced in the database rather than in application logic:

MealVoucher::query()->where('qr_token', ...)->lockForUpdate() inside a transaction (app/Http/Controllers/Api/MealController.php) means that when two scanners submit the same token concurrently, exactly one insert succeeds and the other receives a definitive rejection naming the earlier redemption, per Helland's idempotence argument cited in the proposal's literature review.

- **Sequential re-scans:** 100 of 100 rejected after the first successful redemption.
- **Concurrent races:** 20 of 20 races (five simultaneous scans of one token each) resolved to exactly one success.
- **Cross-category independence:** 20 of 20 reserved attendees, each holding a voucher in all three categories, redeemed every category successfully and were then refused on a repeat of the first. Redeeming breakfast consumes breakfast and nothing else.

Ground truth: 280 redemption rows for 200 batch scans, 20 race winners and 60 cross-category redemptions, matching export_correctness.meals_csv_rows in the evaluation output exactly.

Source: sequential_duplicate_rejection and concurrent_duplicate_rejection in tools/evaluation-results.json.

Multiple meal categories. The second project objective is redemption tracking across multiple meal categories, and both the implementation and the evidence now cover it. MealController exposes create, read, update and delete for categories; a voucher is unique per attendee per category; the redemption record names the category it was redeemed against. The evaluation seeds three categories and issues a voucher per attendee per category, so the 1,500-voucher population the scan batches draw from spans all three.

The property the composite key exists for is measured directly rather than assumed: twenty attendees were reserved before any other step, each holding a voucher in every category, and each had all three redeemed in turn. All sixty redemptions were accepted and every repeat of the first was refused, so redeeming one category neither consumes nor blocks another. The same guarantee is pinned at unit level by

`test_redeeming_one_category_leaves_the_attendees_other_categories_redeemable.`

Source: `cross_category_independence` in `tools/evaluation-results.json`.

2. Session attendance and capacity correctness

The unique (session, attendee) constraint

(`app/Http/Controllers/Api/SessionController.php::scan`) prevents double counting; 20 of 20 duplicate session check-ins were rejected in the evaluation run. Capacity threshold transitions (the configurable warning fraction, default 90%, and over-capacity) push an automatic organiser notification, dispatched after the attendance transaction commits so a push is never sent for attendance that ends up rolled back. This was verified live on a device (`docs/evidence-capacity-alerts.png`) and covered by two automated tests for the alert transitions (`docs/progress-reports/18-audit-fixes.md`).

Source: `session_duplicate_rejection` in `tools/evaluation-results.json`.

3. Latency against the proposal's targets

Nielsen's response-time thresholds, cited in the proposal's literature review, set the sub-second budget for scan-validation round trips at a catering service point where queue throughput matters.

Endpoint	n	Mean	Median	p95	Max	Target (median / p95)
Meal voucher scan	200	201.9 ms	187.5 ms	286.7 ms	347.0 ms	< 500 ms / < 1,000 ms
Session scan	40	186.4 ms	174.5 ms	266.5 ms	292.8 ms	< 500 ms / < 1,000 ms
Analytics summary	20	214.5 ms	211.6 ms	282.6 ms	292.4 ms	(feeds the 30 s freshness bound below)

All measured latencies clear their targets by a wide margin on a local development stack; the proposal's targets were written for venue-grade Wi-Fi, which this local run does not exercise, so these figures are a lower bound rather than a field measurement (see Section 7).

The envelope these numbers hold in. These figures are one named run, and they are a property of that run rather than of the system. Three runs of the identical script against the identical code have now been recorded on this machine: voucher-scan medians of 114.0 ms, 199.0 ms and 187.5 ms, with 95th percentiles of 210.5 ms, 304.3 ms and 286.7 ms. The spread is machine load, not code; the fastest was on an otherwise idle machine and the slowest while a dev server, Word and the document build competed for it.

Across all three runs the correctness results did not move at all: 200 of 200 scans accepted, 100 of 100 sequential duplicates rejected, 20 of 20 concurrent races resolved to exactly one success, and both exports exact against the database.

The honest statement of the envelope is therefore: on a development machine, voucher-scan median latency falls between roughly 110 ms and 200 ms with a 95th percentile between roughly 210 ms and 305 ms, inside the proposal's 500 ms and 1,000 ms targets by a factor of at least three in every run observed; and the correctness guarantees are invariant across runs because they are enforced by database constraints rather than by timing. What is *not* established is behaviour under venue-grade Wi-Fi, under a production multi-worker server, or at attendee counts above 500. Section 7 and the report's limitations state each of those gaps.

Source: voucher_scan_latency, session_scan_latency and analytics_summary_latency in tools/evaluation-results.json.

4. Analytics dashboard freshness

The dashboard polls every 30 seconds (a documented trade-off against introducing a WebSocket layer, proposal Section 8). Worst-case staleness is therefore the poll interval plus the endpoint's own latency: $30\text{ s} + 211.6\text{ ms} \approx 30.2\text{ s}$, within the proposal's $\leq 30\text{ s}$ target when read as "at most one polling cycle plus negligible server time" rather than a strict $\leq 30.000\text{ s}$ bound. This is a property of the chosen architecture, not a measurement that could fail on a different run.

Source: analytics_summary_latency.median_ms in tools/evaluation-results.json.

5. Export correctness

CSV exports are generated by streaming query results directly from the database (app/Http/Controllers/Api/ReportController.php), so export size is bounded by the database rather than server memory, and correctness is a direct row-count comparison rather than a sampled check.

Export	CSV rows	Database ground truth	Match
--------	----------	-----------------------	-------

Export	CSV rows	Database ground truth	Match
meals.csv	280	280 redemptions	exact
attendance.csv	500	500 attendees	exact

Source: export_correctness in tools/evaluation-results.json.

6. Notification delivery

Delivery is recorded per recipient, and this was a defect until it was fixed. The claim this section rests on is that the per-recipient records make delivery auditable, meaning they can answer whether a given person was reached. Until the fix recorded here they could not: the dispatch service applied one aggregate outcome to every recipient row, so a send in which Firebase accepted one device token and refused three still marked every recipient sent, with a delivery timestamp and no failure reason. The refusals were visible only in the application log. The service now maps each token back to the recipient it belongs to: a recipient is sent only when one of that recipient's own tokens was accepted, failed with a reason when all of theirs were refused, and left pending when they have no registered device, which is the honest record for someone who was never pushed to and will receive the message through the in-app inbox instead. The behaviour is pinned by `test_delivery_is_recorded_against_each_recipient_individually`, which fails against the previous implementation.

Live Firebase Cloud Messaging v1 delivery was verified end to end on an Android emulator: an organiser send travelled Laravel → service-account OAuth → FCM v1 → Google → the device's notification shade (docs/evidence-push-delivered.png, docs/progress-reports/15-live-push-delivery.md). In the evaluation run, FCM accepted 100% of valid device tokens submitted; the single stale seeded demo token predictably failed, which is the expected behaviour for an invalid token rather than a defect. Device receipt was observed at approximately 5 seconds on the one test device used; a multi-device fleet measurement of the proposal's 95%-within-30-seconds target is recorded as future work rather than claimed here. Because FCM delivery is best-effort, every notification is also persisted to an authenticated in-app inbox, giving 100% eventual delivery to any attendee who opens the app, independent of push reachability.

Source: docs/progress-reports/15-live-push-delivery.md, docs/progress-reports/19-evaluation-run.md.

7. Baseline comparison, and its limits

The proposal's evaluation plan calls for comparing the voucher module against a simulated, human-staffed paper-register meal line. That live human-trial baseline was **not** conducted: organising a staffed comparison exercise was not feasible within the project's scope. This is

stated plainly rather than approximated, because a fabricated baseline number would be worse than admitting the gap.

What the evaluation does establish is the system-side bound: a median scan-to-decision time of 187.5 ms, with single-use enforcement that a paper register cannot provide at any speed (a photographed or reused paper voucher has no server-side check to fail). The paper frames the comparison qualitatively on this basis rather than quoting a fabricated throughput figure for the manual condition.

8. Reproducibility

8.1 The environment these commands assume

PHP 8.5.8 for Windows, with the `pdo_pgsql` and `pgsql` extensions enabled in its `php.ini`, along with `openssl`, `curl`, `mbstring`, `zip`, `gd` and `fileinfo`. The PostgreSQL client library `libpq.dll` ships beside the executable. A PHP without those extensions loaded cannot connect to the database and none of the commands below will run.

The build used here is a portable one, installed to a directory on the user `PATH`, so the commands are the ordinary `php artisan ...` a reader would expect. For a machine where PHP is not on `PATH`, the repository also carries `php-pgsql.bat`, a one-line wrapper that resolves to a PHP under a `.tools` directory beside the repository; substituting `.\php-pgsql.bat` for `php` makes every command below work unchanged. `serve-pgsql.bat` wraps it again to clear the config cache and serve on port 8000. The interpreter itself is not committed, because a platform-specific binary distribution does not belong in a source repository, but its version and required extension set are recorded here so the environment can be rebuilt rather than guessed at.

The rest of the toolchain, at the versions the committed deliverables were built against: PostgreSQL 18.2, Python 3.14.0, pandoc 3.9.0.1, and Microsoft Word 16.0, which `tools/paginate_report.py` drives to export the PDFs and so to measure the page numbers printed in the contents list. The Python packages the document build imports are pinned in `tools/requirements-docs.txt` and installed with `pip install -r tools/requirements-docs.txt`. The evaluation runner `tools/run-evaluation.py` imports only the standard library and needs none of them.

8.2 Commands

The full evaluation round, against a local backend:

```
php artisan serve
php artisan tinker tools/seed-evaluation-event.php
python tools/run-evaluation.py
```

This regenerates `tools/evaluation-results.json` from the seeded dataset. It overwrites the recorded run, so the figures quoted in this report come from the named run committed in

that file rather than from whatever run happened last; Section 3 states the spread observed across runs.

Backend test suite:

```
php artisan test
```

Static analysis:

```
php vendor/bin/phpstan analyse --memory-limit=1G
php vendor/bin/pint --test
```

PHPStan analyses in parallel worker processes that each receive PHP's `memory_limit`, 128 MB by default, and on this project a worker exhausts that limit often enough that the default cannot be relied on: the run dies with a fatal error instead of producing a report. It succeeded on a cleared result cache on an idle machine and failed on two other occasions when the machine was busier. Passing `--memory-limit=1G` makes it deterministic, and that is the form recorded above. The condition is a property of the analyser's memory use, not a defect in the code: whenever the analysis completes it reports zero errors at level 7.

Mobile:

```
flutter analyze
flutter test
```

Word and PDF deliverables, rebuilt from this markdown and from the report:

```
python tools/build_docx_deliverables.py
python tools/paginate_report.py
```

`tools/paginate_report.py --check` fails if the page numbers printed in the contents lists no longer match the rendered documents.